

Code management and git servers

Vladislav Goncharenko



METU + Yandex,
spring 2026

Recap

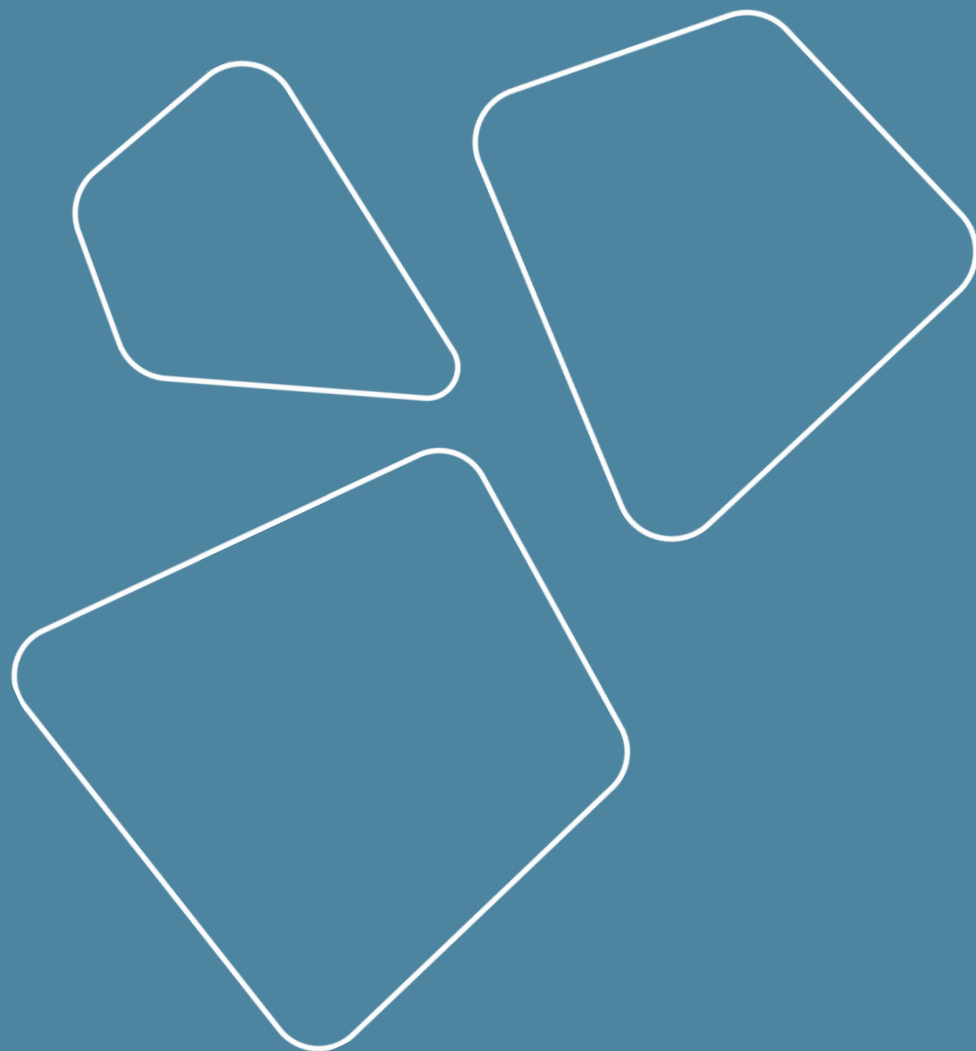
- Зачем нужен MLOps
- Стандарты разработки Data Science проектов
- Что такое MLOps
- Подзадачи MLOps
- Обзор решений от крупных компаний
- Проблемы open source стека

Practice:

- Пакетирование в Питоне
- Управление зависимостями
 - poetry
- Jupyter server с паролем

Outline

1. Code management
2. Git servers
3. Workflow
 - a. Distribution
4. CI/CD
5. Codebase structure
 - a. Code formatting
 - b. Inline documentation
6. pre-commit
7. code quality tools



Code management

girafe
ai

01

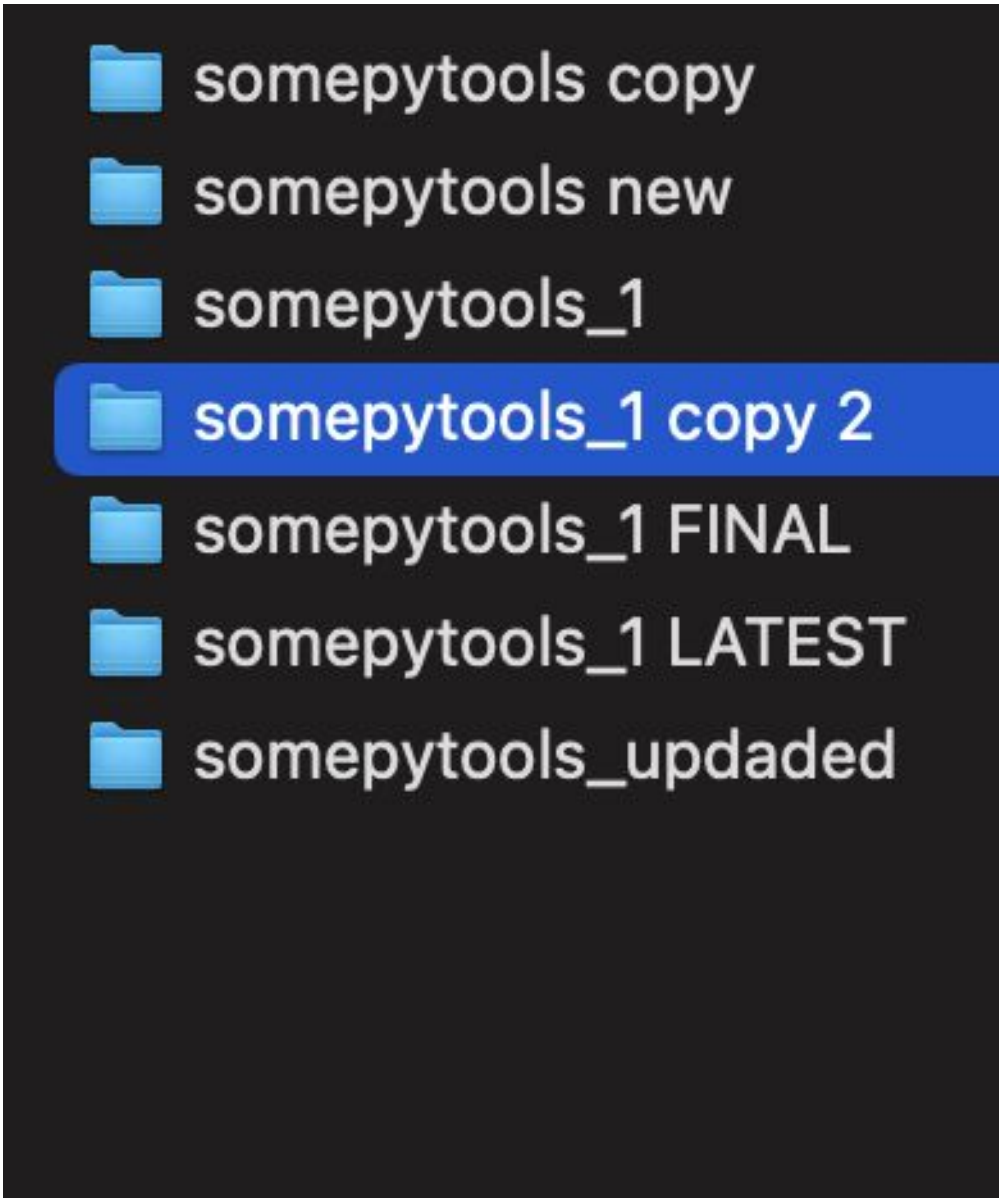
Code management

- One local copy
- Several local copies

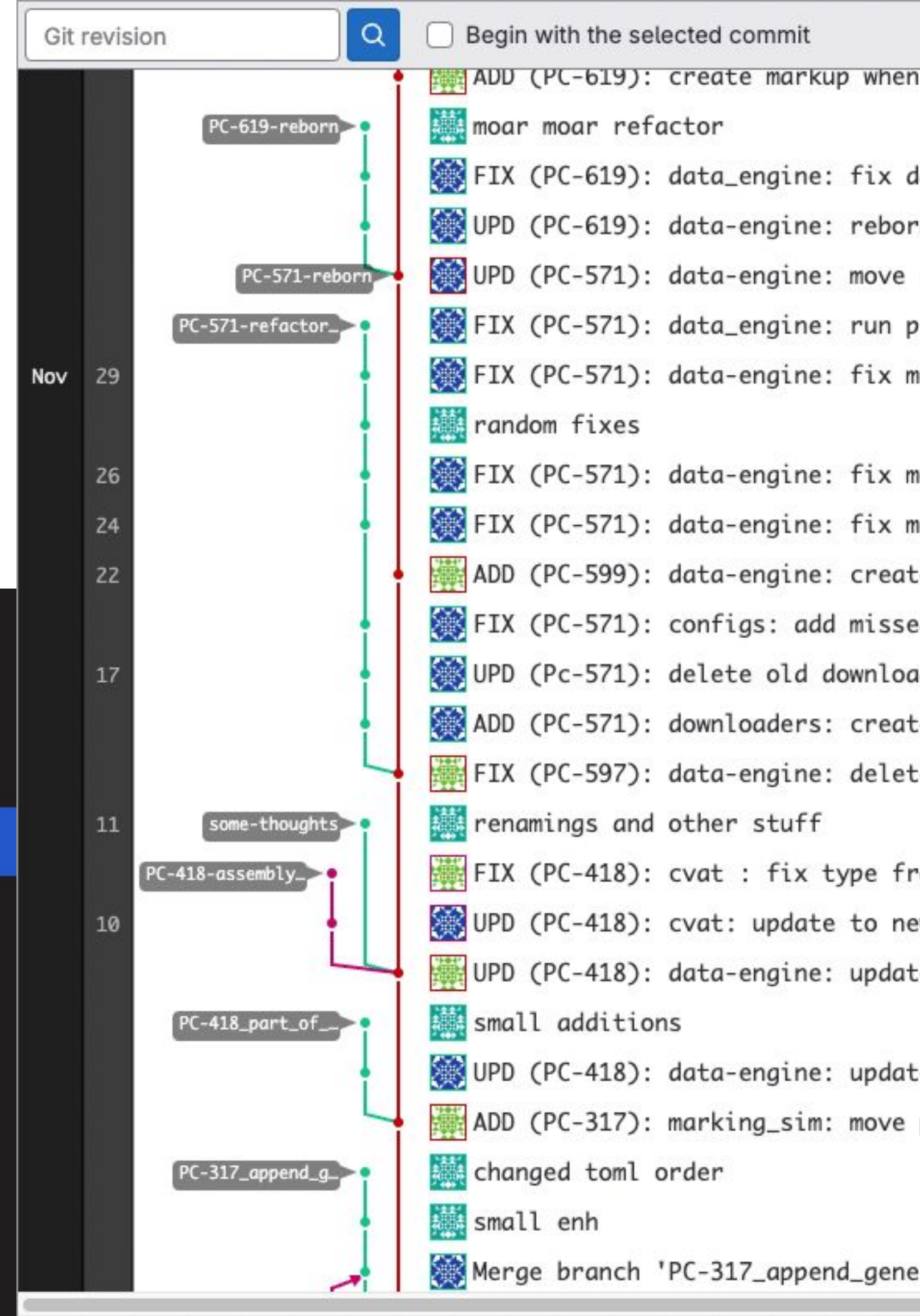


Code management

- One local copy
- Several local copies
- Remote copies
- Version control system (VCS)



somepytools copy
somepytools new
somepytools_1
somepytools_1 copy 2
somepytools_1 FINAL
somepytools_1 LATEST
somepytools_updated



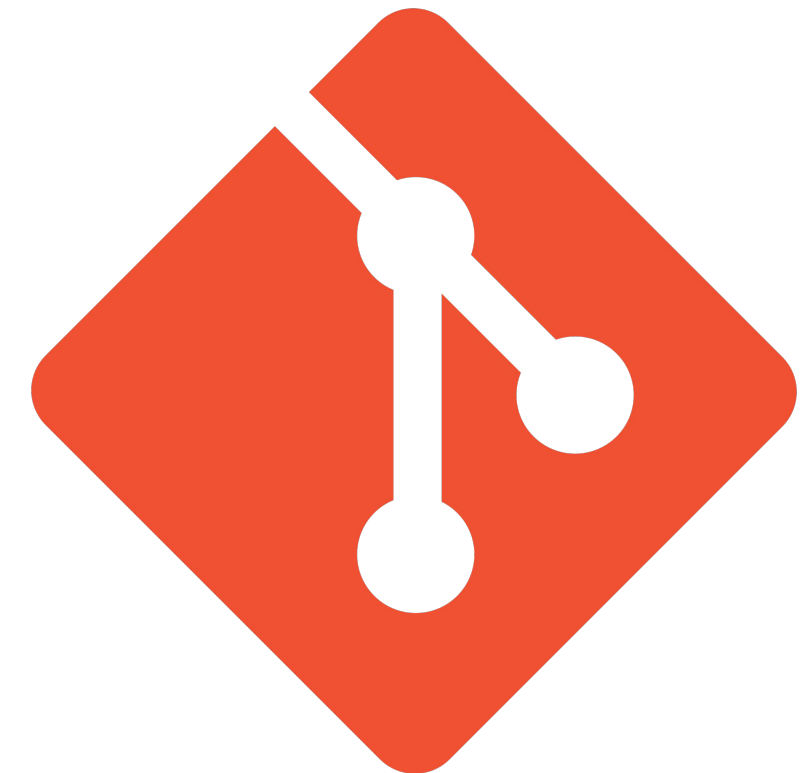
Version control systems (VCS)

- Local only
 - [Source Code Control System](#) (SCCS) – part of UNIX [1973]
- Client–server
 - Apache Subversion (SVN) [2000]
 - Yandex Arcadia (fork of SVN)
 - master branch is called trunk



Version control systems (VCS)

- Local only
 - [Source Code Control System](#) (SCCS) [1973]
- Client-server
 - [Apache Subversion](#) (SVN) [2000]
 - Yandex Arcadia (fork of SVN)
- Distributed
 - [Git](#) [2005]
 - [Mercurial](#) [2005]
 - [Plastic](#) (.NET) [2006]
 - [Perforce](#) [1995]

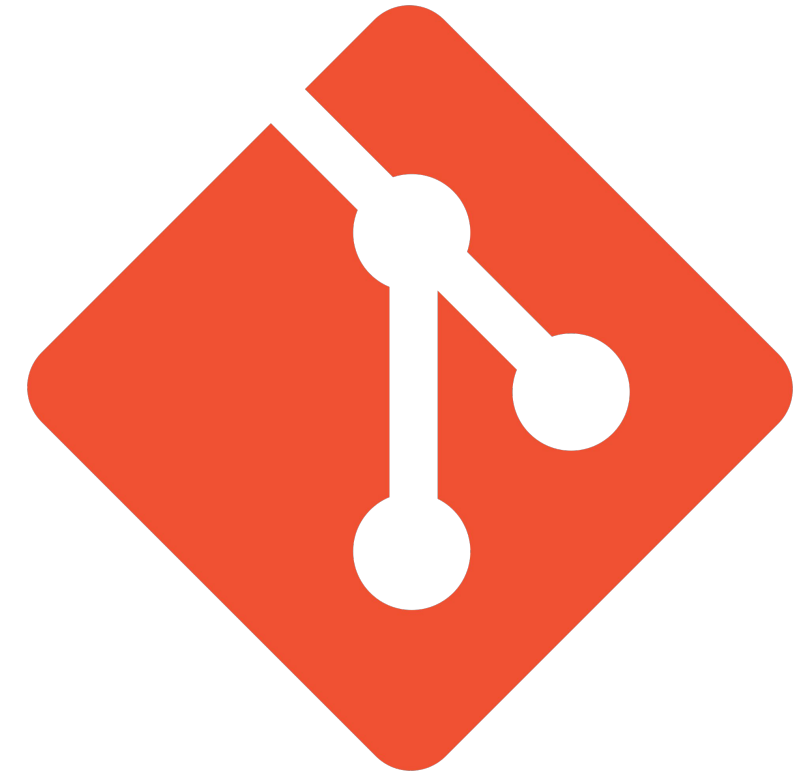


mercurial

PERFORCE

Git is not a silver bullet

- <https://graphite.com/blog/why-facebook-doesnt-use-git>
-



Learn git

Brilliant git tutorial by Atlassian:

<https://www.atlassian.com/git/tutorials/what-is-version-control>

Covers topics from basics to advanced.
Can be read cover to cover.

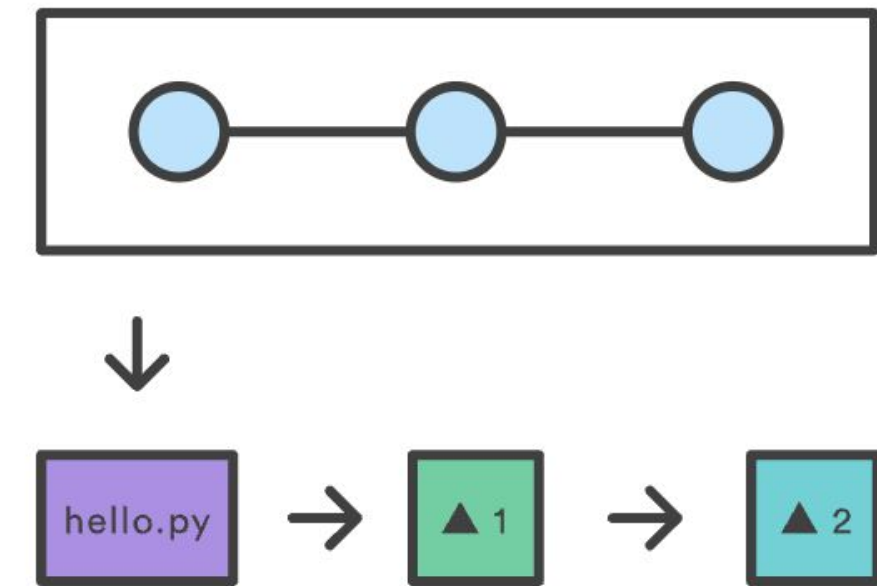
Complete git reference <https://git-scm.com/book/en/v2>

Short explanation of crucial concepts

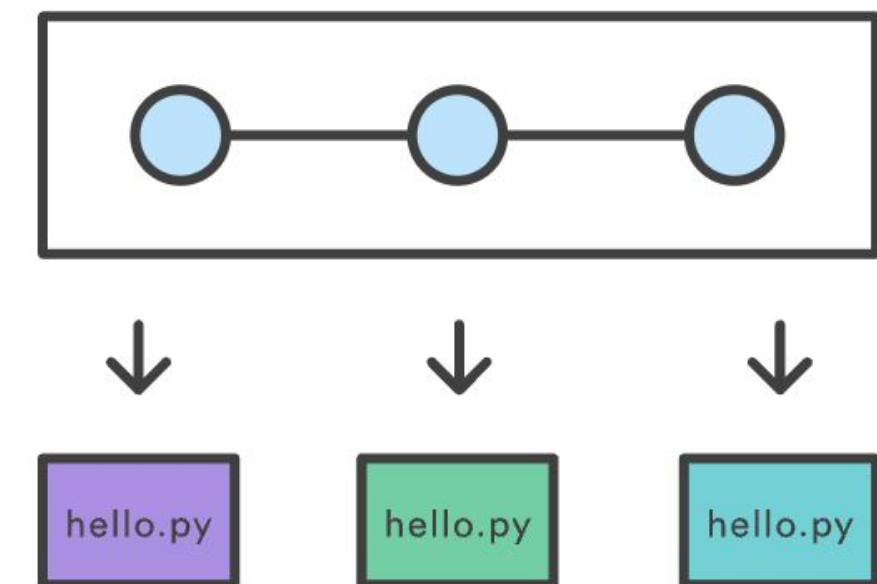
<https://m.youtube.com/watch?v=Ala6PHIYjmw>

Interactive tutorial <https://learngitbranching.js.org/>

Recording File Diffs (SVN)



Recording Snapshots (Git)



Git forges

- [SourceForge](#) [1999]
 - pioneer
- [GitHub](#) [2008]
 - proprietary
 - [owned by Microsoft](#) since 2018
- Atlassian [Bitbucket](#) [2008]
- [GitLab](#) [2011]
 - open source, but not free
 - introduced integrated CI/CD
- [gitee](#) [2013]
 - Chinese GitHub
- [Gogs](#) [2014]
 - FOSS
 - example of running service - [GIN](#)
- [Gitea](#) [2016] -> [Forgejo](#) [2022]
 - fork of Gogs FOSS, integrated CI/CD. <https://codeberg.org/> works on it



GitHub

Founded in 2008 as a free-for-open-source (but proprietary) platform. It have shown rapid growth over the next ten years.

In 2018 [acquired by Microsoft](#), [launched Actions](#).

In 2020 [GitHub acquired npm](#)

GitHub made git main SVC of XXI century, so it became corporation

Still it has many issues such as heavy Actions, some corporate policies and so on:

- <https://ziglang.org/news/migrating-from-github-to-codeberg/>
 - <https://janikvonrotz.ch/2025/08/20/migrate-from-github-to-codeberg/>
- https://www.youtube.com/watch?v=E3_95BZYIVs



Superstars of Github

as of July 2026

Rank	Name	Stars	Forks	Description
1	build-your-own-x	~500k	~50k	Master programming by recreating your favorite technologies from scratch.
2	awesome	~490k	~36k	Awesome lists about all kinds of interesting topics
5	free-programming-books	~400k	~66k	Freely available programming books
8	system-design-primer	~360k	~57k	Learn how to design large-scale systems. Prep for the system design interview. Includes Anki flashcards.
9	coding-interview-university	~360k	~84k	A complete computer science study plan to become a software engineer.

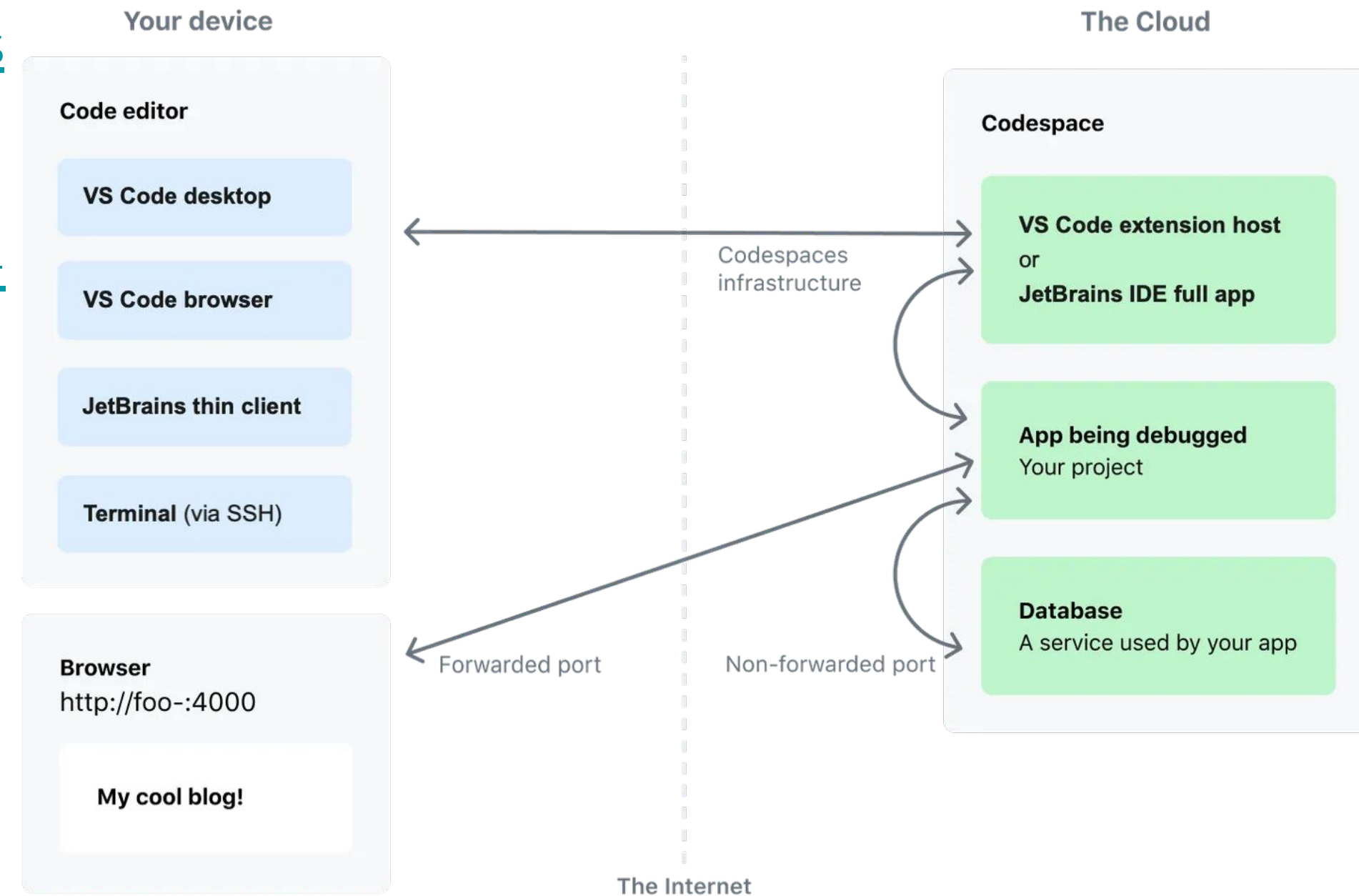
<https://github.com/EvanLi/Github-Ranking/blob/master/Top100/Top-100-stars.md>

Online IDE with Github

<https://github.com/features/codespaces>

<https://docs.github.com/en/codespaces/developing-in-a-codespace/creating-a-codespace-for-a-repository>

<https://docs.github.com/en/codespaces/about-codespaces/deep-dive>



Badges in README.md on github

What to store in repo

- Use gitignore - it is mandatory
 - global: exclude OS dependent files
 - e.g. .DS_store for macOS
 - local: exclude project and language specific files
 - e.g. pycache, data files
 - [taken from github repo](#)
- Code
- Configs

Not to store:

- Data
- Model artifacts
- Visualizations
- Jupyter Notebooks

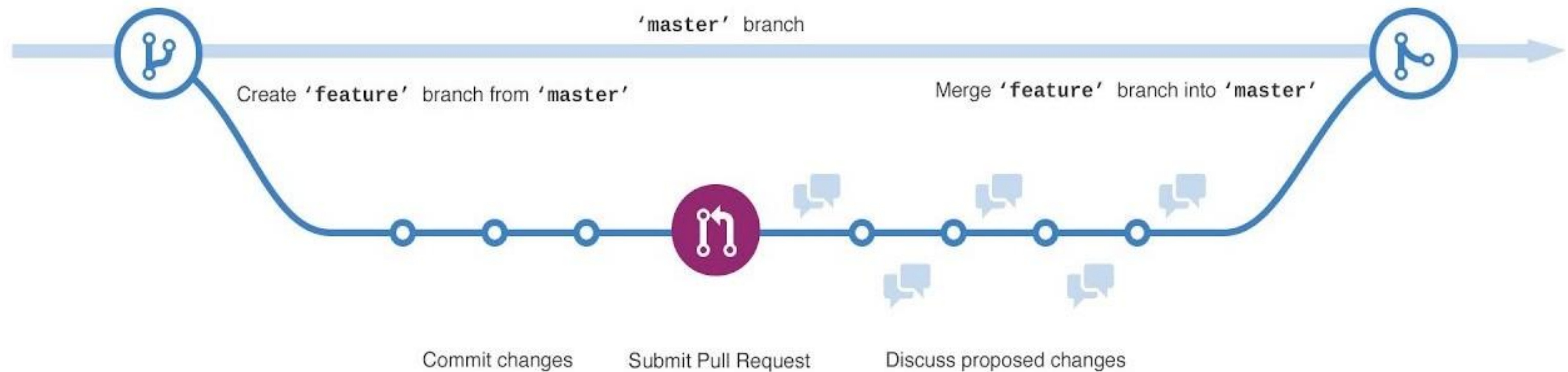
Git workflow

girafe
ai

02

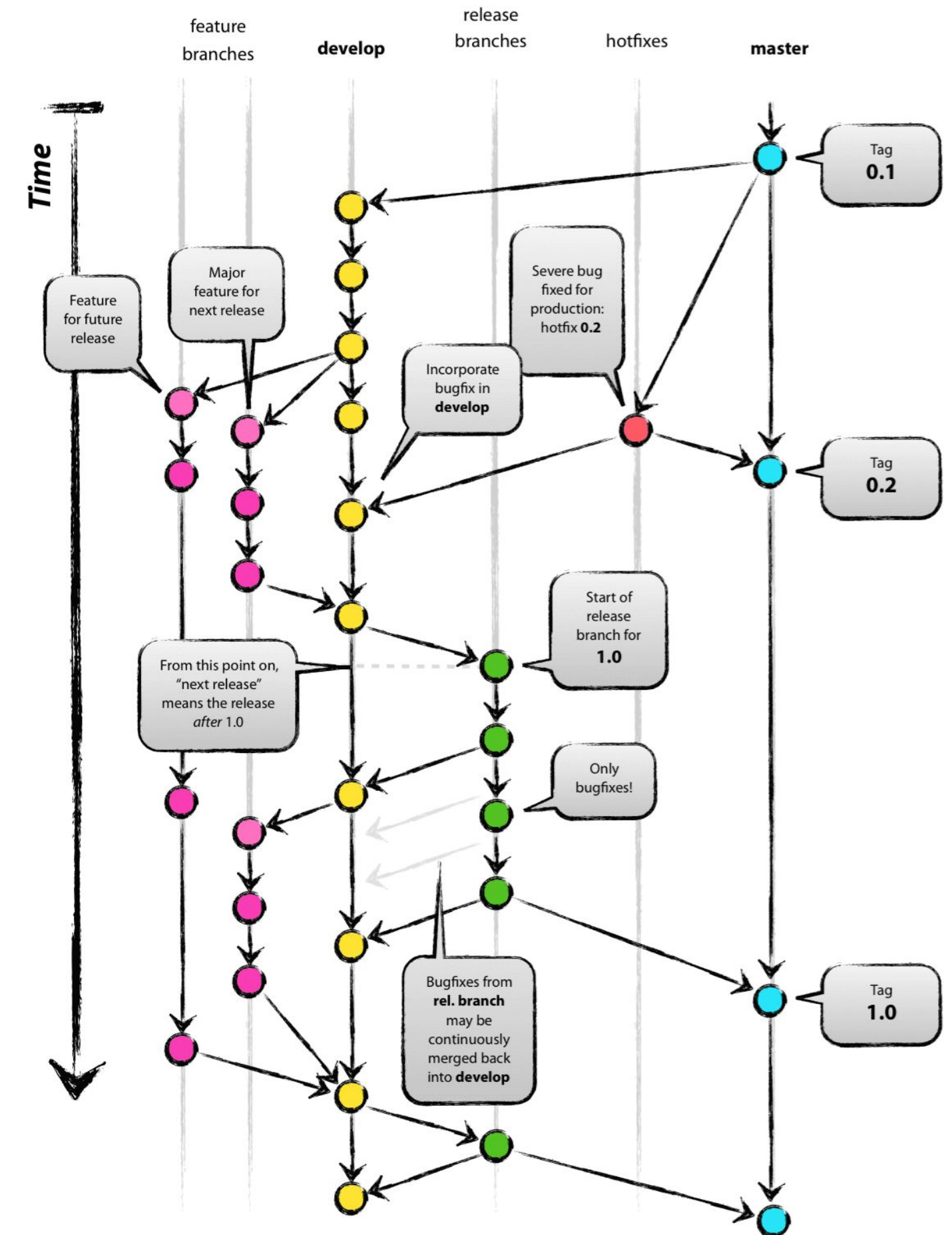
Teamwork

- Best practice - [merge requests \(pull requests\)](#)

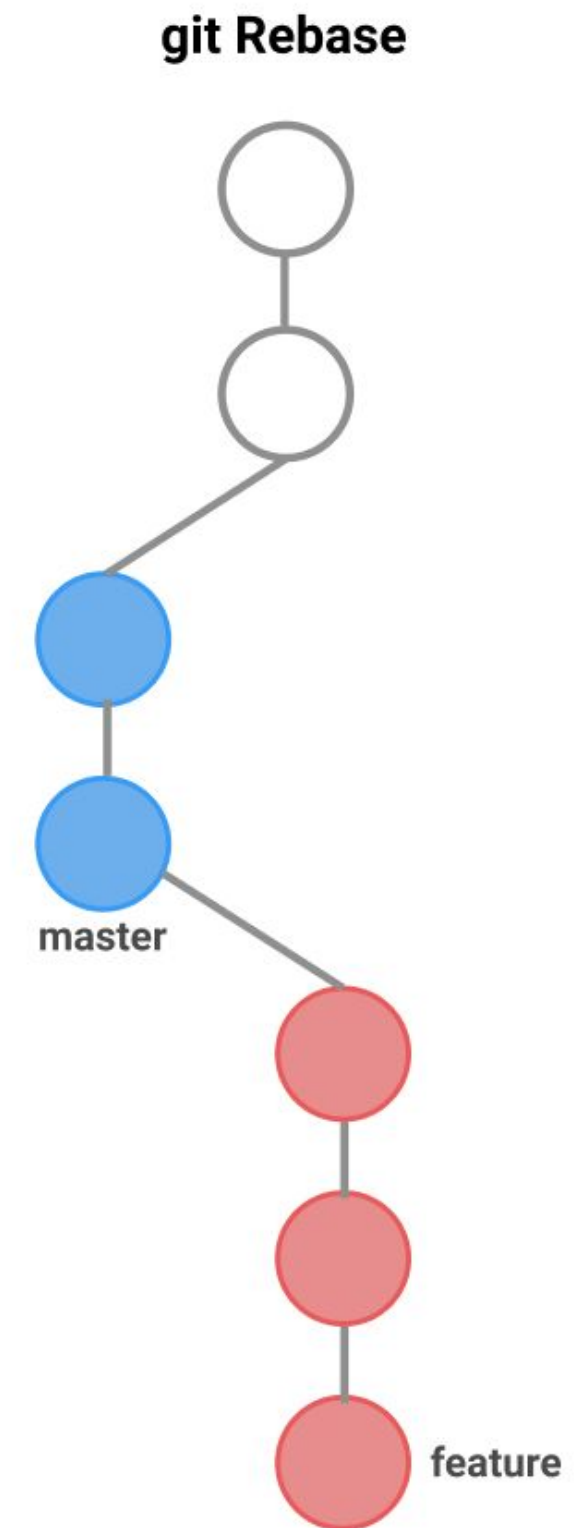
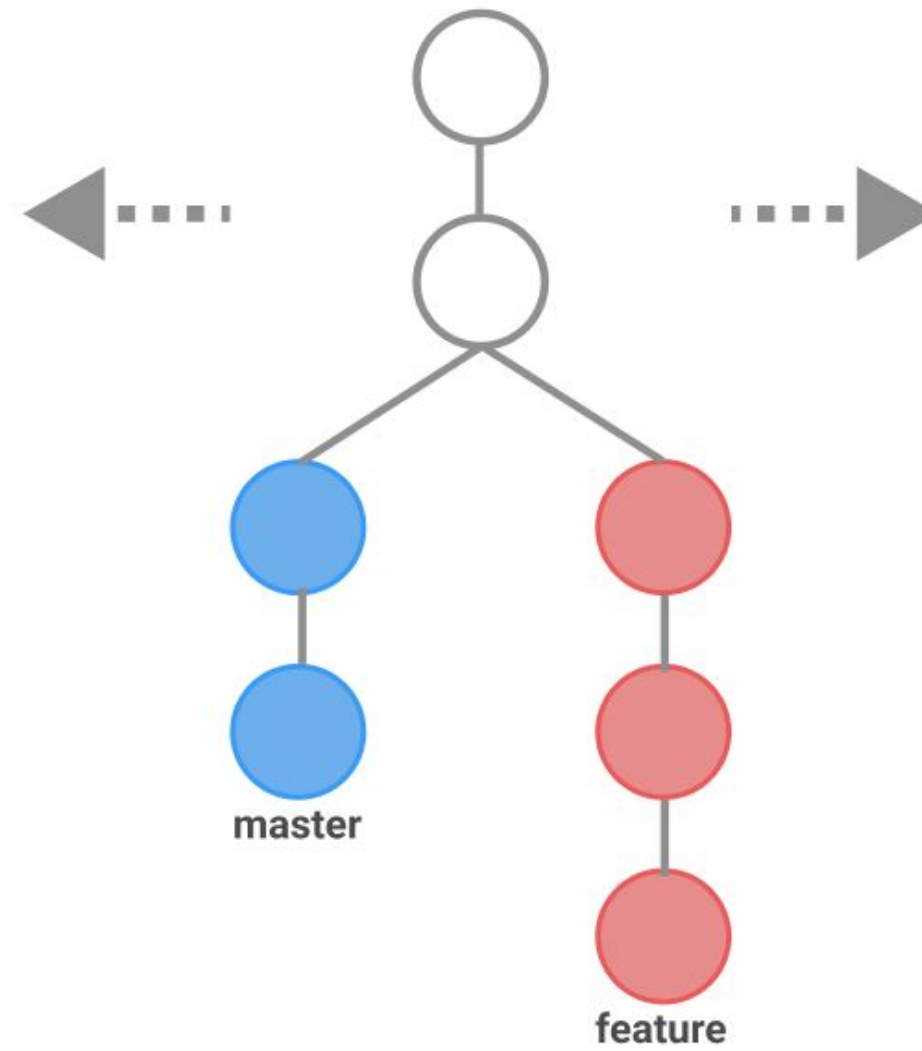
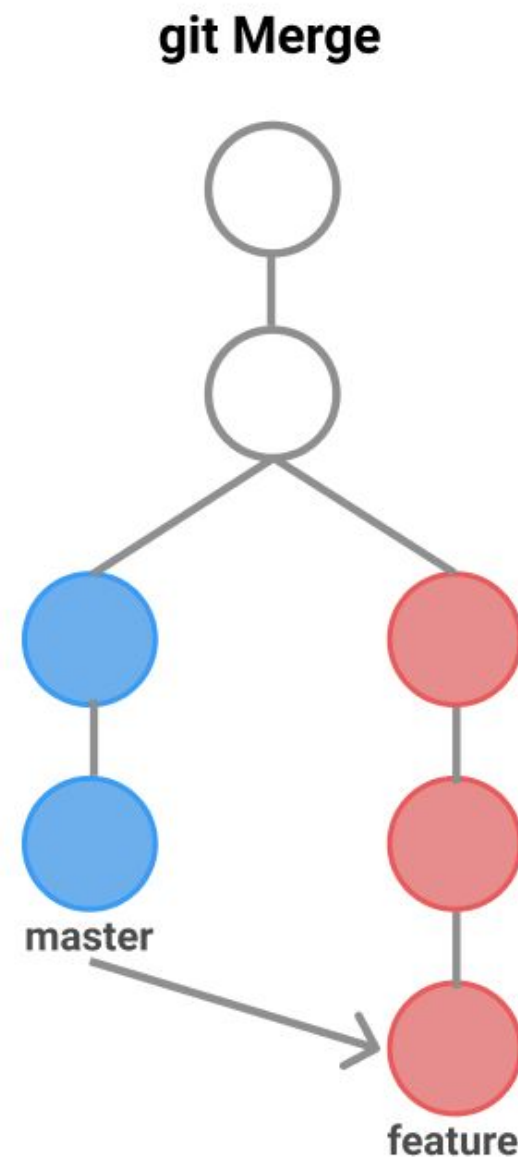


Teamwork

- Best practice - [merge requests \(pull requests\)](#)
 - Advanced - [Git Flow \(original article\)](#) [2010]
 - [Flows comparison](#)



Git rebase



Git client implementations

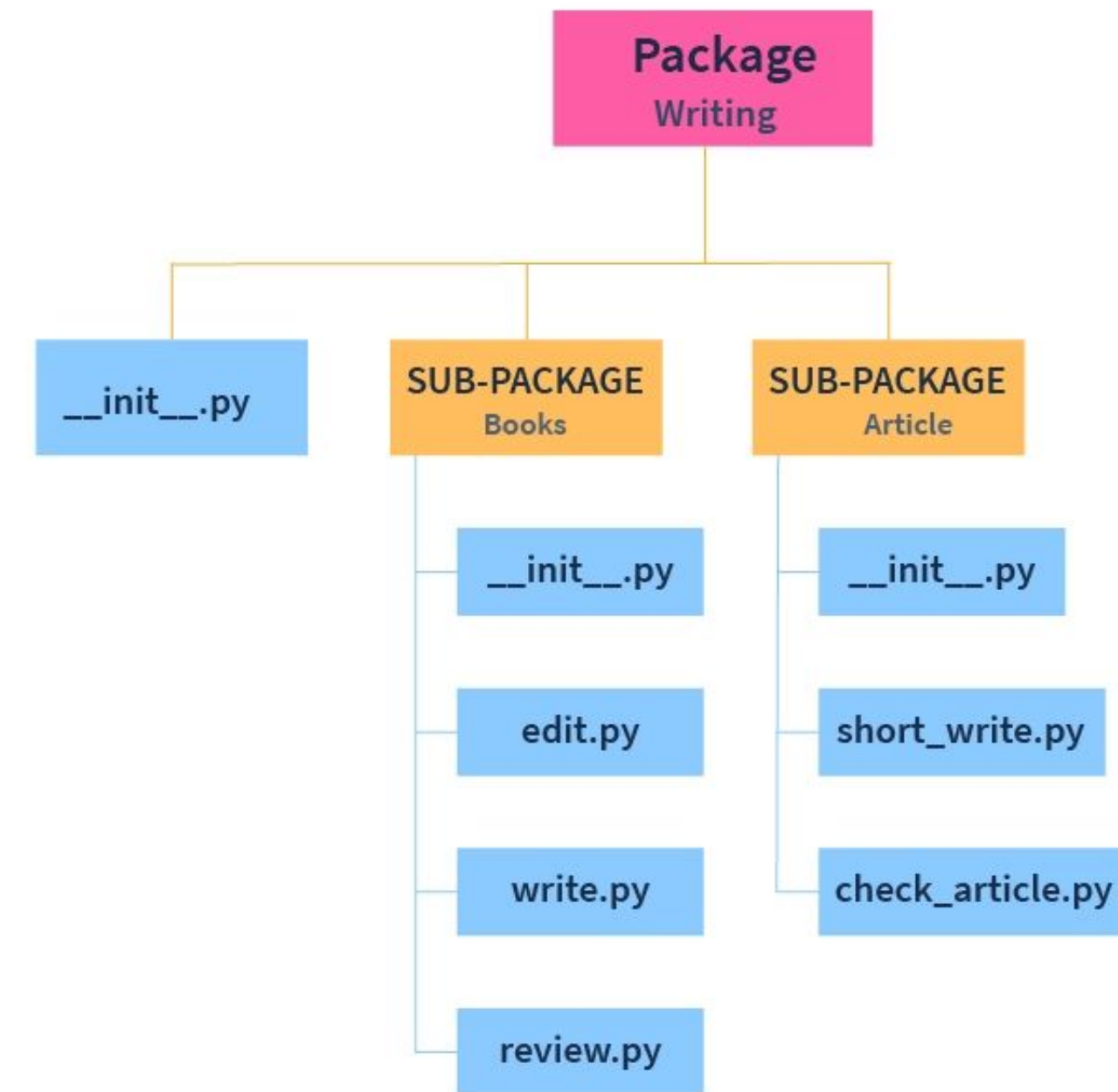
- on JavaScript <https://isomorphic-git.org/>
-

Git parent search algorithm

-

Distribution

- Source code
 - clone git repo to final machine
 - update with git pull
 - doesn't need structure
- Packages
 - install with pip install
 - update with pip update
 - requires structure
 - for simple packaging use [poetry](#)
- Special tools
 - ONNX
 - TensorRT
- Special services
 - Triton inference server
 - MLflow serving
 - Seldon MLServer



Continuous Integration (CI) & Continuous Delivery (CD)

girafe
ai

03

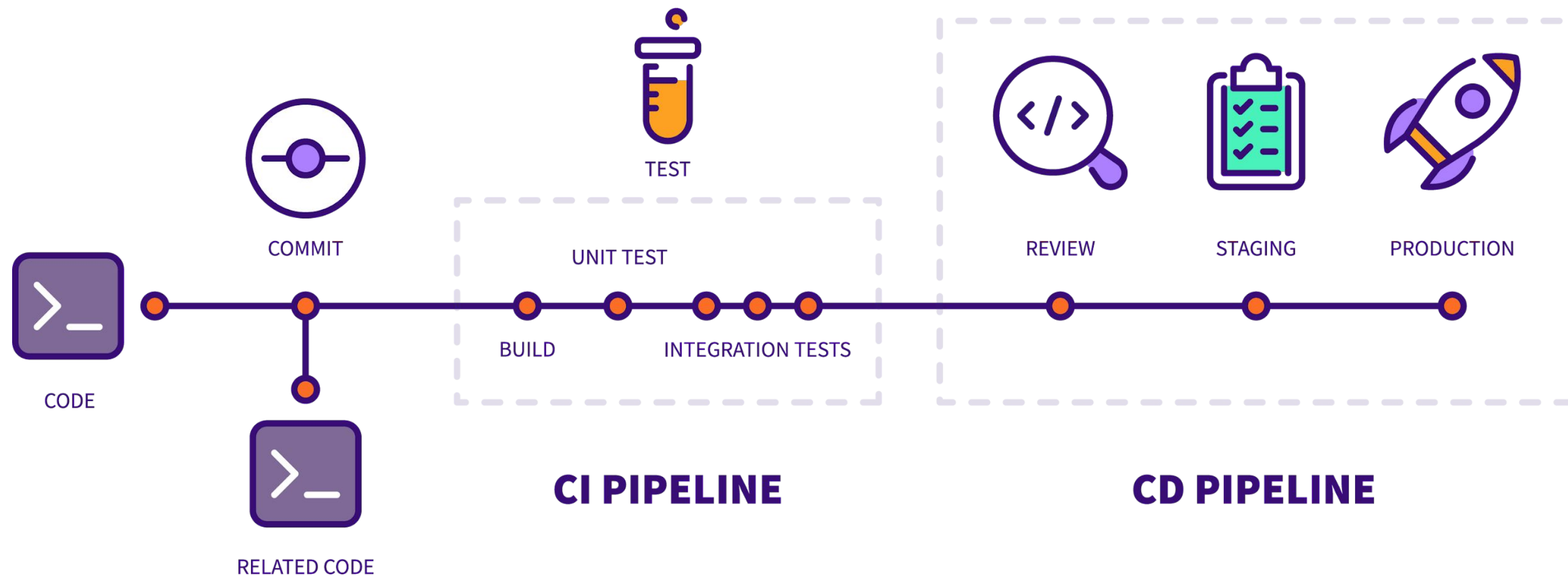
CI/CD stacks

Git server integrated

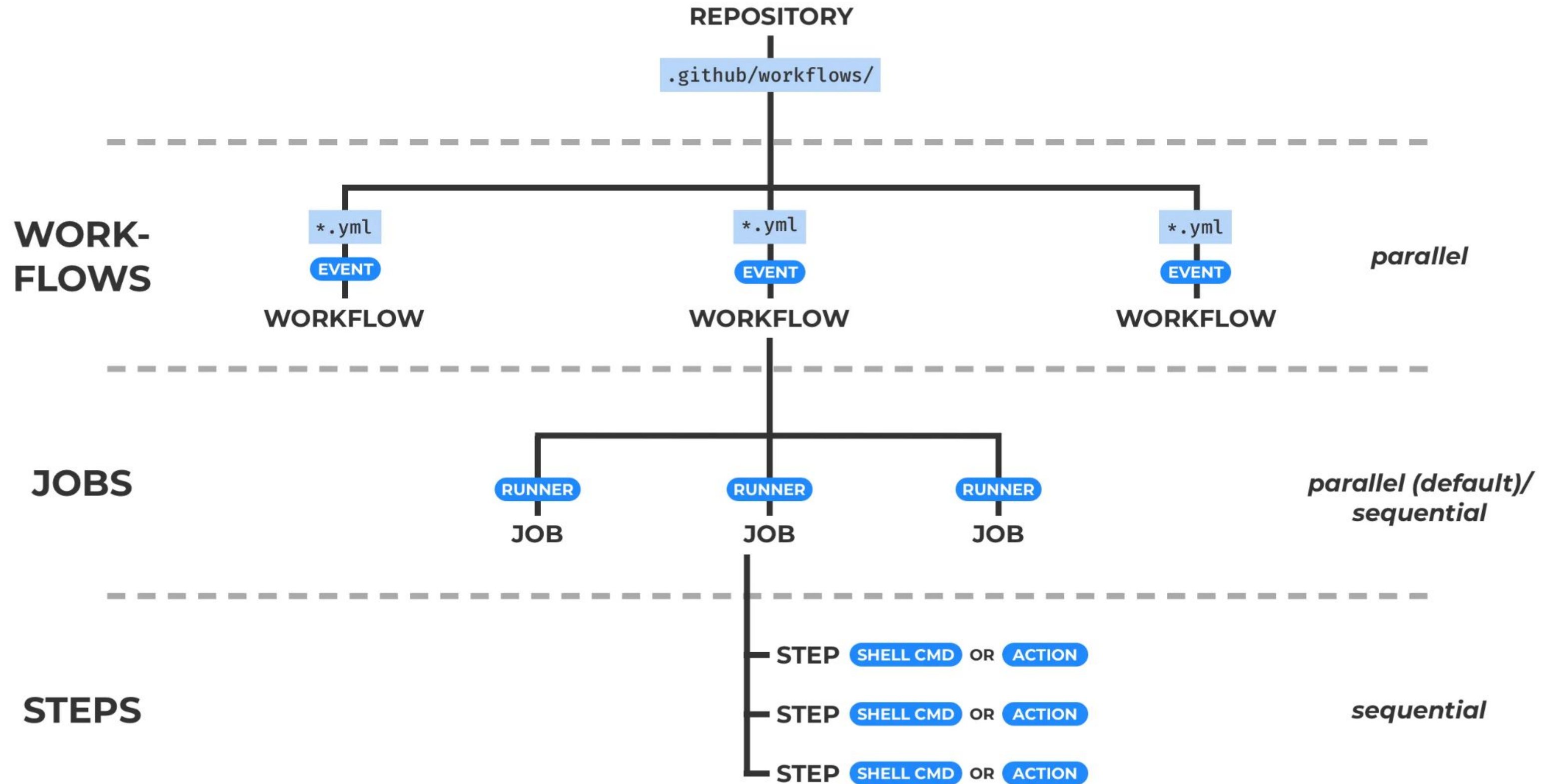
- Github Actions
- GitLab CI
- Gitea Actions
- Atlassian Bamboo

Standalone

- Jenkins
- Travis
- Circle CI
- TeamCity
- [AgroCD](#)



Github Actions entities



A price of using CI

- <https://www.feldera.com/blog/the-pain-that-is-github-actions>

Especially bad case is Github Actions which are Azure pipelines under the hood

<https://www.youtube.com/watch?app=desktop&v=9qljpi5jiMQ>

Code quality tools

girafe
ai

04

The Zen of Python

```
>>> import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Explicit is better than implicit.

[Simple is better than complex.](#)

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

Code styles

- Python standards
 - [PEP 8](#) (code style), [PEP 257](#) (docstrings)
- Custom standards
 - [Google Python Style Guide](#)
 - [Black code style](#) (rationale)

A Foolish Consistency is the Hobgoblin of Little Minds

One of Guido's key insights is that code is read much more often than it is written. The guidelines provided here are intended to improve the readability of code and make it consistent across the wide spectrum of Python code. As [PEP 20](#) says, "Readability counts".

A style guide is about consistency. Consistency with this style guide is important. Consistency within a project is more important. Consistency within one module or function is the most important.

However, know when to be inconsistent – sometimes style guide recommendations just aren't applicable. When in doubt, use your best judgment. Look at other examples and decide what looks best. And don't hesitate to ask!

Typing styles

- [snake_case](#)
 - used for most things in Python
- [CamelCase](#) aka PascalCase
 - used for class names in Python
- UPPER_CASE
 - constants
- [kebab-case](#)
 - used for repo names on Github, Gitlab, etc



Code formatters

[Black](#)

The uncompromising code formatter

[PyCon talk](#)

Other solutions:

- [yapf](#) - configurable
- [autopep8](#) - only pep8
- [ruff](#) - fast rust implementation



```
class Foo (    object ):
    def f    (self
    ):
        return        37*-2
    def g(self, x,y=42):
        return y
def f (    a: List[ int ]):
    return        37-a[42-u : y**3]
```

```
class Foo(object):
    def f(self):
        return 37 * -2

    def g(self, x, y=42):
        return y

def f(a: List[int]):
    return 37 - a[42 - u : y ** 3]
```

Imports formatter

- [isort](#)

Sorts import to 5 categories:

- future
- standard-library
- third-party
- first-party
- local-folder



```
import os
from math import ceil
import myproject.test
```

```
import .utils
from math import sqrt
import django.settings
```

```
from math import ceil, sqrt
import os
```

```
import django.settings
```

```
import myproject.test
```

Linters

- [flake8](#)
 - [List of rules explained](#)
 - light and modular
- [pylint](#)
 - cross-file analysis

Example of rules:

- ❖ F401: Module imported but unused
- ❖ E402: Module level import not at top of file
- ❖ F811: Redefinition of unused name from line n

More solutions from [Python Code Quality Authority](#):

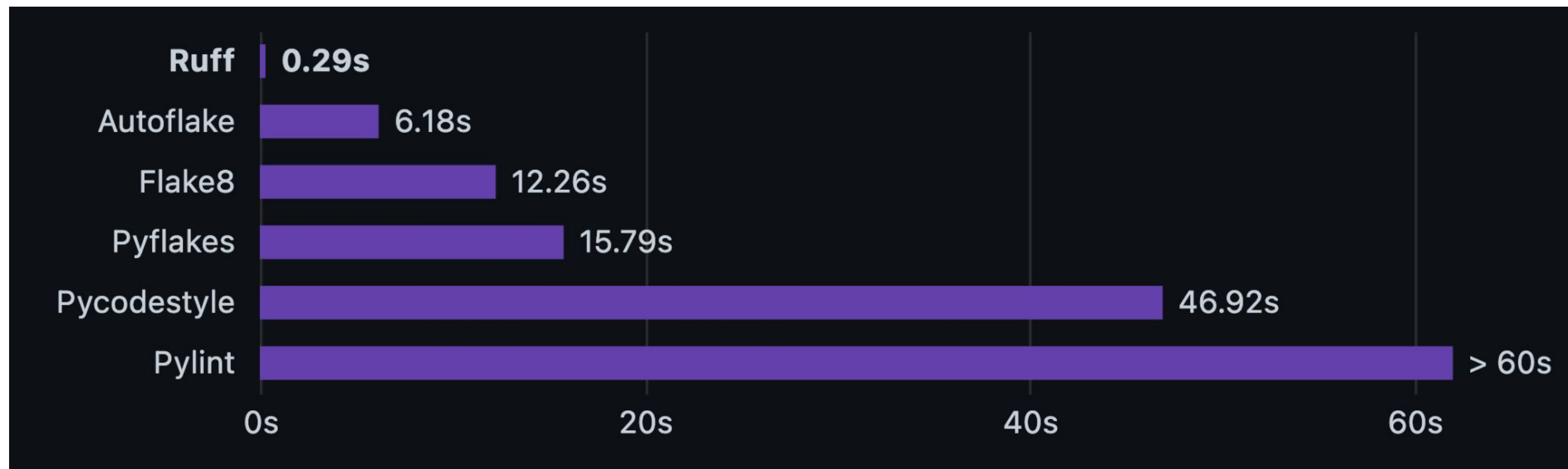
- [bandit](#)
- lots of them...



Python code quality on rust

Ruff

- <https://github.com/astral-sh/ruff>
- <https://docs.astral.sh/ruff/rules/>
- <https://docs.astral.sh/ruff/linter/>



Typing for Python



Best for code editors, not pre-commit

- [mypy](#)
- [Pyright](#)
- [Pyre](#) / [pyrefly](#)
- [ty](#) (from Astral)

Seamless dynamic and static typing

From Python...

```
def fib(n):  
    a, b = 0, 1  
    while a < n:  
        yield a  
        a, b = b, a+b
```

...to statically typed Python

```
def fib(n: int) -> Iterator[int]:  
    a, b = 0, 1  
    while a < n:  
        yield a  
        a, b = b, a+b
```

Code quality for Jupyter notebooks

- [nbQA](#)
Same functionality for Jupyter Notebooks!
Supports pre-commit



nbQA
Quality Assurance For Jupyter Notebooks

Code style for other languages

- [prettier](#)
 - YAML
 - TOML
 - Markdown
 - Dockerfile
 - bash
 - SQL
 - etc

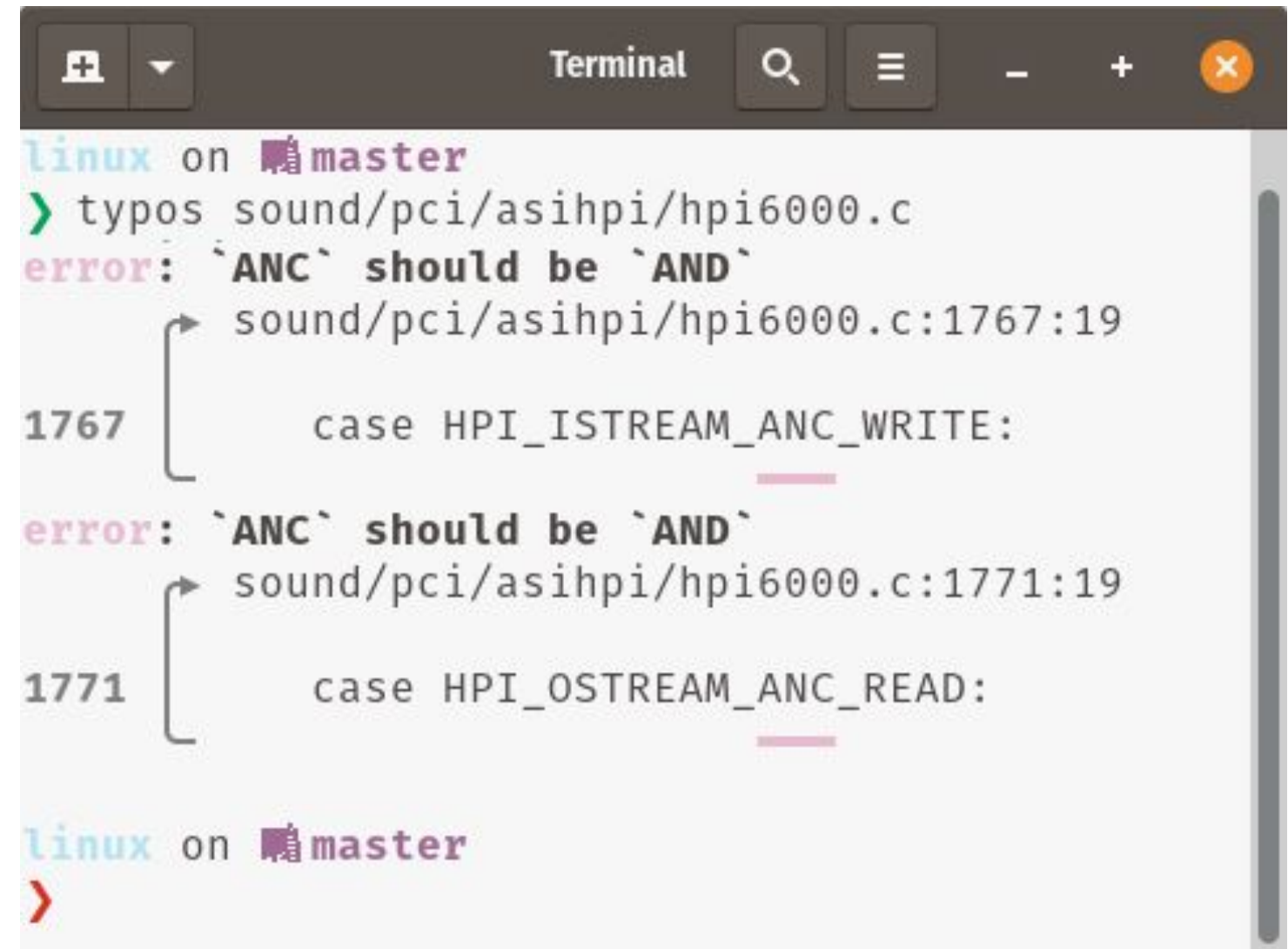


Prettier

Spell checking

Two main paradigms are presented nowadays:

- known misspelling → known correction
 - Low false positives, good for CI
 - [typos](#): modern Rust implemented
 - [codespell](#): weaker Python implemented
- token not in dictionary → potential misspelling
 - Higher recall, noisier, good for texts and docs
 - [cspell](#): TypeScript implemented

A terminal window titled "Terminal" with a dark background. It shows the command "typos sound/pci/asihpi/hpi6000.c" being executed. The output shows two errors: "error: `ANC` should be `AND`" at line 1767 and line 1771. The errors are highlighted in red, and the corresponding code lines are also highlighted. The terminal shows the file path "sound/pci/asihpi/hpi6000.c" and the line numbers "1767" and "1771". The code snippets are "case HPI_ISTREAM_ANC_WRITE:" and "case HPI_OSTREAM_ANC_READ:". The terminal also shows the prompt "linux on master" and a red prompt character ">".

```
linux on master
> typos sound/pci/asihpi/hpi6000.c
error: `ANC` should be `AND`
    sound/pci/asihpi/hpi6000.c:1767:19
1767  [ case HPI_ISTREAM_ANC_WRITE:
      [
error: `ANC` should be `AND`
    sound/pci/asihpi/hpi6000.c:1771:19
1771  [ case HPI_OSTREAM_ANC_READ:
      [
linux on master
>
```

Pre-commit

[pre-commit](#)

uses [git hooks](#) to run common utilities before git commands processing
not only for Python and not only for code quality!



```
$ pre-commit install
pre-commit installed at /home/asottile/workspace/pytest/.git/hooks/pre-commit
$ git commit -m "Add super awesome feature"
black.....Passed
blacken-docs.....(no files to check)Skipped
Trim Trailing Whitespace.....Passed
Fix End of Files.....Passed
Check Yaml.....(no files to check)Skipped
Debug Statements (Python).....Passed
Flake8.....Passed
Reorder python imports.....Passed
pyupgrade.....Passed
rst ``code`` is two backticks.....(no files to check)Skipped
rst.....(no files to check)Skipped
changelog filenames.....(no files to check)Skipped
[master 146c6c2c] Add super awesome feature
1 file changed, 1 insertion(+)
```

Pre-commit

- pre-commit is development dependency
 - enables you to avoid code quality tools as dependencies (even development)
- creates separate envs for each tool itself
 - you don't need to add code quality tools as dependency!
 - works even for non-Python dependencies (e.g. npm, rust)
- you may use any [git hook](#) through this tool (not only pre-commit)
- writing your own hooks is very simple 😊 [example](#)

Alternatively one may use [husky](#) to run git hooks, but it's JS world...

Code quality IDE integration

You may want to have code quality tools [directly in your IDE](#).

Main recommendation is ruff for vscode: <https://github.com/astral-sh/ruff-vscode>

Configuring it to apply rules on save will make your life easier!

Other tools also have their integrations with most popular IDEs

```
// Use Ruff as default formatter for Python
"ruff.enable": true,

"[python]": {
  "editor.codeActionsOnSave": {
    "source.fixAll": "explicit",
    "source.organizeImports": "explicit"
  },
  "editor.defaultFormatter": "charliermarsh.ruff",
  "editor.formatOnSave": true
},
```

Alternative: Makefile

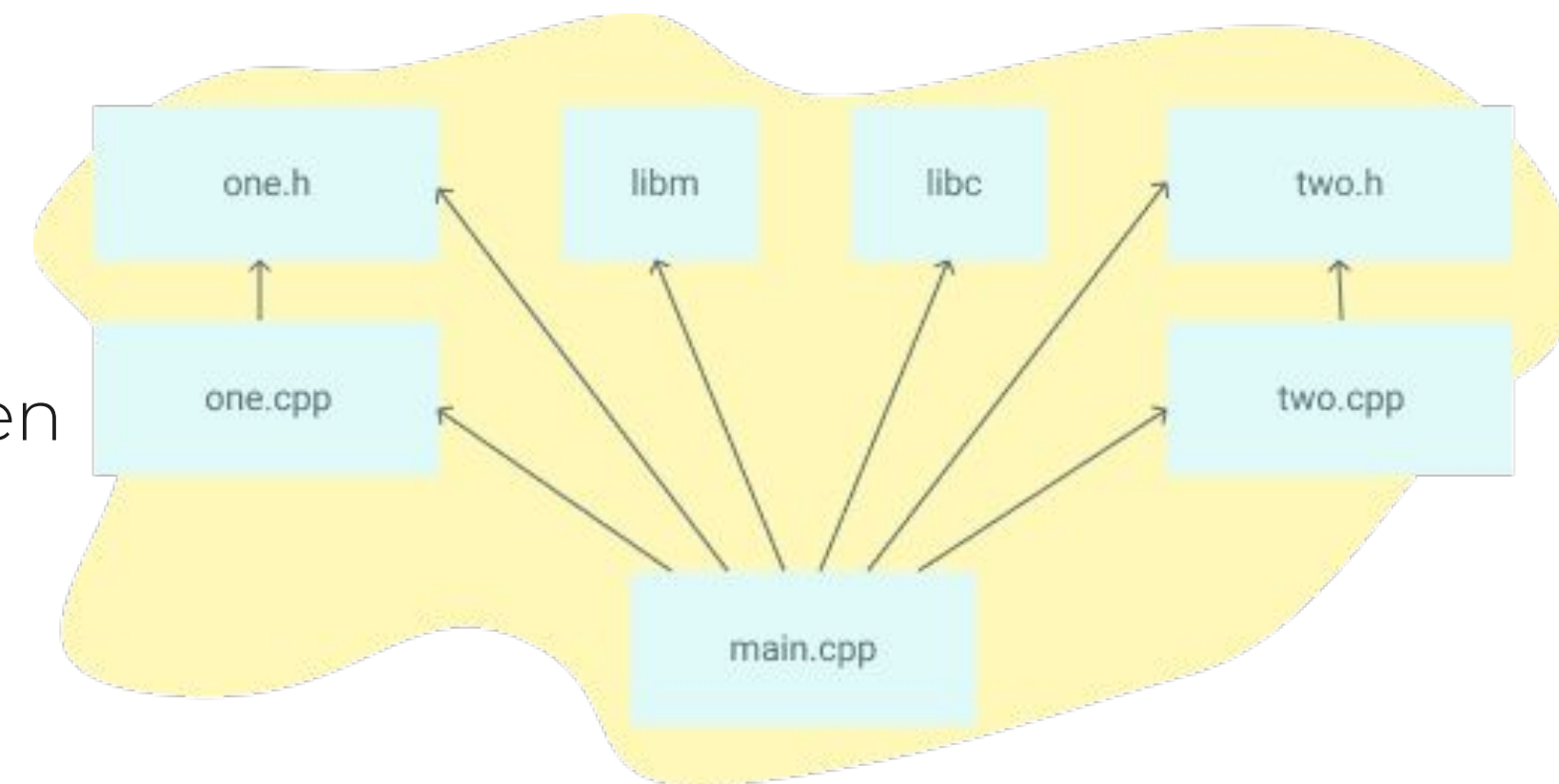
<https://makefiletutorial.com/>

Makefile in context of code quality:

- main goal of Make is building executables (not code quality tools running)
- pre-commit is made for code quality tools
- pre-commit makes separate envs itself
- pre-commit uses git diffed files, not all
- for pre-commit Windows is a first class citizen

So pre-commit is more suitable to run code quality pipelines.

<https://www.youtube.com/watch?v=299Lc10v7hU>



Standard library tools

- [annotations](#) aka type hints
- [typing module](#)
- [pathlib](#)

Standard library tools

- [annotations](#) aka type hints
- [typing module](#)
- [pathlib](#)
- [logging](#)
 - [tutorial](#)

Documentation Driven Development

Whole idea behind something driven development is first make something and only then write code.

Test driven development requires considerable effort on test writing.

But documentation is usually needed for any project, so you can write it first.

[Learn More](#)



Documentation generation

- [docstrings](#)
- [Sphinx](#)
- [mkdocs](#)
- [mkdocs-material](#)
-



consists in two steps

- parsing (docstring style)
 - Google format
 - [numpy format](#)
 - ...
 - [see uv docs on that](#)
- rendering (visual style)
 - pytorch
 - read the docs
 - [numpy](#)

Documentation for every project

<https://deepwiki.com/>

Is able to create documentation for a project even there's no official one.

Tests and coverage

- [pytest](#)
 - [pytest-cov](#)
- <https://docs.python.org/3/library/doctest.html>

<https://sourcery.ai/blog/python-best-practices/>

Revise

- Хранение кода
- Git servers
- Разработка в больших командах
 - Дистрибуция кода
- CI/CD
- Форматирование кода
- Inline документация

Practice:

- pre-commit
- инструменты контроля качества

Спасибо за внимание!

Вопросы?

